

1. Operating Systems & Networks

1.1. Lab Worksheet 11

The Application Layer — HTTP & Apache — the HTTP Request/Response Cycle Kathara edition

1.2. This Lab's Objectives

1. Understand the structure of a Kathara lab.
2. Launch and manage a small multi-machine Kathara topology.
3. Observe and log HTTP communication between two virtual machines.
4. Analyse the HTTP request/response cycle.
5. Capture and inspect HTTP and TCP traffic with `tcpdump` and Wireshark.

1.3. Expected Outcomes

After this laboratory session, students should be able to:

- create and launch a multi-machine Kathara lab;
- connect to running Kathara machines from the host;
- configure and test a simple client/web-server lab;
- capture HTTP traffic with `tcpdump`;
- inspect captured traffic with Wireshark;
- explain the steps in a complete HTTP request/response cycle.

1.4. Workplan

These lab worksheets assume that you make accurate notes of everything you do for later revision. Record:

- commands used;
- the purpose of each command;
- output and observations;
- answers to the questions posed in the worksheet.

You should complete any unfinished work before the next lab session.

1.5. Starting Assumptions

You should already be able to:

- use basic Linux file and directory commands;
- start and stop Kathara labs;
- inspect network interfaces and addresses;
- use `ping`, `tcpdump`, and Wireshark;
- navigate between the host system and virtual machines;
- save files into the Kathara machine overlay directories.

Note: This worksheet assumes you are using Debian-based Kathara machines with the following tools available:

- `apache2`
- `apache2ctl`
- `tcpdump`
- `lynx`
- `telnet` OR `nc`
- `nmap`

If any of these are missing from your image, install them first or use a prepared image supplied by your tutor.

1.6. Lab Layout and Files

A Kathara lab directory for this exercise will typically contain:

- one `lab.conf` file;
- one startup file per machine;
- one directory per machine, used as the filesystem overlay;
- optional subdirectories inside each machine directory for configuration files, web content, and captures.

For this lab we will use two machines:

- `web-client`
- `web-server`

1.7. 1. Create the Lab Directory

On the host system:

```
cd ~  
mkdir -p ctec1704/lab_11  
cd ctec1704/lab_11
```

Create the machine directories:

```
mkdir -p web-client web-server
```

1.8. 2. Create the Kathara Lab Files

Create `lab.conf`:

```
gedit lab.conf &
```

Add the following content:

```
web-client[0]=lan  
web-server[0]=lan
```

Create `web-client.startup`:

```
gedit web-client.startup &
```

Add:

```
ip addr add 192.168.150.1/24 dev eth0  
ip link set eth0 up
```

Create `web-server.startup`:

```
gedit web-server.startup &
```

Add:

```
ip addr add 192.168.150.2/24 dev eth0
ip link set eth0 up
```

1.9. 3. Check the Directory Structure

List the directory contents:

```
ls -l
```

You should now have something similar to:

```
lab.conf
web-client/
web-client.startup
web-server/
web-server.startup
```

You can also use tree:

```
tree
```

Insert image here

Example tree output for the lab directory.

1.10. 4. Review the Configuration Files

Use cat to inspect each file:

```
cat lab.conf
cat web-client.startup
cat web-server.startup
```

Record the following in your notes:

- machine names;
- interface names;
- IP addresses;
- the shared network segment.

Draw a simple network diagram showing:

- web-client ON 192.168.150.1/24
- web-server ON 192.168.150.2/24
- both connected to lan via eth0

1.11. 5. Start the Lab

From the lab_11 directory on the host:

```
kathara lstart
```

Kathara starts the machines in the background. Unlike Netkit, this does not usually open xterm windows automatically.

Open two host terminals and connect to each machine:

```
kathara connect web-client
```

In another host terminal:

```
kathara connect web-server
```

Note: If your local setup uses a slightly different workflow, for example wrapper scripts or an IDE integration, follow your local Kathara procedure for opening terminals on each machine.

1.12. 6. Test Connectivity

From web-client, test connectivity to the server:

```
ping -c 2 192.168.150.2
```

From web-server, test connectivity back to the client:

```
ping -c 2 192.168.150.1
```

Record the results.

1.13. 7. Starting the Apache Web Server

In Kathara containers, systemd is often not running, so this guide uses apache2ctl rather than systemctl.

From web-server, first check whether Apache is already running:

```
ps -ef | grep '[a]pache2'  
ss -ltnp | grep ':80'
```

If Apache is not running, start it:

```
apache2ctl start
```

Check again:

```
ps -ef | grep '[a]pache2'  
ss -ltnp | grep ':80'
```

Question: What is a PID in this context?

Write your answer in your notes.

Insert image here

Apache running on the web-server machine.

1.14. 8. Inspect the Default Web Page

On web-server, locate the default web root:

```
ls -l /var/www/html
cat /var/www/html/index.html
```

This directory is the web root of the site.

Record:

- the path to the default page;
- the file name of the default page;
- a short description of its contents.

1.15. 9. Access the Web Page with Telnet

From web-client, connect to TCP port 80 on the web server:

```
telnet 192.168.150.2 80
```

If telnet is unavailable, use:

```
nc 192.168.150.2 80
```

You should see a connection message.

Now type the HTTP request manually:

```
GET / HTTP/1.0
```

Then press Enter twice to terminate the request headers with a blank line.

You should receive an HTTP response similar to:

```
HTTP/1.1 200 OK
Date: ...
Server: Apache/...
Last-Modified: ...
ETag: ...
Accept-Ranges: bytes
Content-Length: ...
Connection: close
Content-Type: text/html

<html> ... </html>
```

Analyse the separate parts of the request:

- GET
- /
- HTTP/1.0

Then analyse each response line, especially:

- the status line;
- Date;
- Server;
- Content-Length;
- Connection;
- Content-Type.

1.16. 10. Apache Control Commands

From web-server, restart Apache if needed:

```
apache2ctl restart
```

To stop Apache:

```
apache2ctl stop
```

To start it again:

```
apache2ctl start
```

1.17. 11. Remove the Apache ServerName Warning

Apache may warn that it cannot reliably determine the server's fully qualified domain name.

In a Debian-style Apache setup, the cleanest fix is to add a config snippet under `conf-enabled`.

Exit both Kathara machine consoles, then on the host create:

```
mkdir -p web-server/etc/apache2/conf-enabled  
\  
gedit web-server/etc/apache2/conf-enabled/servername.conf &
```

Put the following into the file:

```
ServerName 127.0.0.1
```

Restart the lab so the new overlay file is used:

```
kathara lclean  
kathara lstart
```

Reconnect to web-server and start Apache again:

```
kathara connect web-server
```

Inside the machine:

```
apache2ctl start
```

Check whether the warning has disappeared.

1.18. 12. Start Apache Automatically at Boot

Edit `web-server.startup` on the host and add `apache2ctl start` below the IP commands:

```
gedit web-server.startup &
```

The file should now look like this:

```
ip addr add 192.168.150.2/24 dev eth0
\  
ip link set eth0 up
\  
apache2ctl start
```

Restart the lab:

```
kathara lclean
kathara lstart
```

Reconnect to `web-server` and check whether Apache is already running:

```
kathara connect web-server
```

Inside the machine:

```
ps -ef | grep '[a]pache2'
ss -ltnp | grep ':80'
```

From `web-client`, check that you can still download the default page.

1.19. 13. Other HTTP Methods

From `web-client`, connect again to the server:

```
telnet 192.168.150.2 80
```

Test the following methods one at a time:

```
POST / HTTP/1.0
```

```
HEAD / HTTP/1.0
```

```
TRACE / HTTP/1.0
```

For each one:

- press Enter twice after typing the request line;

- record the response;
- compare it with the GET response.

Question: How does the output from HEAD differ from the output from GET?

Question: What does TRACE return?

Now repeat with deliberate errors, for example:

```
get / HTTP/1.0
GeT / HTTP/1.0
GET /missingpage HTTP/1.0
GET / HTTP/9.9
```

Observe how the response changes in each case.

1.20. 14. Capture an HTTP Request/Response Cycle

Create a captures directory in the server overlay on the host:

```
mkdir -p web-server/captures
```

Start a packet capture on web-server:

```
kathara connect web-server
```

Inside the machine:

```
tcpdump -v -i eth0 -s 0 -w /captures/http1.pcap
```

Leave tcpdump running.

1.21. 15. Use the Lynx Text-Based Browser

From another host terminal, connect to web-client:

```
kathara connect web-client
```

Inside web-client, launch lynx:

```
lynx 192.168.150.2
```

You should see the default web page in text form.

Insert image here

lynx showing the default web page.

Question: Who might need a text-based browser, and why?

Exit lynx.

If you are unsure how to exit, try:

```
Press q, then confirm.
```

Return to the web-server terminal and stop tcpdump with Ctrl+C.

Open the capture on the host using Wireshark:

```
wireshark web-server/captures/http1.pcap &
```

1.22. 16. Analyse the Capture in Wireshark

There should be roughly a small TCP conversation, often around a dozen packets.

Identify the packets that contain:

- the TCP three-way handshake;
- the HTTP request;
- the HTTP response;
- the response payload;
- the TCP connection close.

In Wireshark, inspect the TCP header and note the flag settings in:

- SYN
- SYN/ACK
- ACK
- FIN
- FIN/ACK

Insert image here

Wireshark analysis of the HTTP request/response cycle.

Record:

- source and destination IP addresses;
- source and destination port numbers;
- sequence of protocol events;
- where the HTTP payload appears.

1.23. 17. Replace the Default index.html

On the host, create your own page in the server overlay:

```
mkdir -p web-server/var/www/html  
\  
gedit web-server/var/www/html/index.html &
```

Use this content:

```
<html>
\
<body>
\
<h1>It works!</h1>
\
</body>
\
</html>
```

Now extend it by adding a paragraph:

```
<html>
\
<body>
\
<h1>It works!</h1>
\
<p>Hello there!</p>
\
</body>
\
</html>
```

Restart the lab so the updated file is used:

```
kathara lclean
kathara lstart
```

Use `lynx` from `web-client` to confirm that your edited file is being served.

1.24. 18. Add a Second Web Page

Create a second page on the host:

```
gedit web-server/var/www/html/page2.html &
```

Use:

```
<html>
\
<body>
\
<h1>This is page 2</h1>
\
<a href="index.html">Return to the home page</a>
\
</body>
\
</html>
```

Now edit `index.html` and add a link to the second page:

```
gedit web-server/var/www/html/index.html &
```

Updated content:

```
<html>
\
<body>
\
<h1>It works!</h1>
\
<p>Hello there!</p>
\
<a href="page2.html">Page 2</a>
\
</body>
\
</html>
```

Restart the lab again:

```
kathara lclean
kathara lstart
```

Use lynx to:

- open the home page;
- follow the link to page 2;
- return to the home page.

Repeat the packet capture procedure while you navigate between both pages.

Then inspect the new .pcap in Wireshark and identify the complete request/response cycle for both pages.

1.25. 19. Use nmap

Create a new capture on the server:

```
mkdir -p web-server/captures
```

Connect to web-server and start:

```
tcpdump -v -i eth0 -s 0 -w /captures/nmap1.pcap
```

From web-client, scan the web server:

```
nmap -v -sV 192.168.150.2
```

Be patient while the scan completes.

Read the output carefully and note:

- open ports;
- detected services;
- version information.

Stop the capture and open it in Wireshark:

Wireshark web-server/captures/nmap1.pcap &

Insert image here

nmap output from the client scanning the web server.

Analyse the capture, especially:

- the handshake packets;
- resets, if any;
- scans against closed ports;
- how the HTTP service is identified.

Question: What do the `-v` and `-sV` switches do?

Question: Are there any other `nmap` switches that would give useful traffic for analysis?

Use `man nmap` to help answer this.

1.26. 20. Shut Down the Lab

When you have finished:

Kathara lclean

Check that the lab has stopped cleanly.

1.27. 21. Review

Review what you learned in this lab session. You should now be able to explain:

- how Kathara represents a small network lab;
- how HTTP runs over TCP;
- the structure of a basic HTTP request;
- the structure of an HTTP response;
- how Apache serves files from the web root;
- how packet captures reveal both transport-layer and application-layer behaviour.

1.28. Follow-Up Work

Look up common HTTP header fields and explain:

- why each one is needed;
- example values for each;
- how a web browser uses them to request and receive a web page.

You should also be able to explain, step by step, how a browser requests a page and how a server replies.

For modern HTTP reading, use the current HTTP specifications and your module resources. If your course materials still reference older HTTP RFCs, compare them with newer specifications and note the differences in terminology and structure.

Make sure you have completed all steps in this worksheet before the next lab session.